Kalman Filter - Explained from Scratch

What is a Kalman Filter?

When trying to track a moving object, there are two sources of information:

1. **The motion model** - knowing where the object was and how fast it was moving makes it possible to predict where it is now.
2. **The sensor** - the camera measured the object at a certain position, but the measurement is noisy.

Both are imperfect. The Kalman filter is an algorithm that **optimally combines these two noisy sources** every single frame to produce the best possible estimate of where the object is.

Notation

Symbol	Full Name	Meaning
$\mathbf{s}$	State vector	Everything being tracked (position and velocity)
$\mathbf{P}_{\text{cov}}$	Covariance matrix	How confident the filter is about each part of the state
$\mathbf{F}_{\text{trans}}$	Transition matrix	Encodes the motion model (how state evolves over time)
$\mathbf{H}_{\text{obs}}$	Observation matrix	Maps the state to what the sensor can actually measure
$\mathbf{Q}_{\text{proc}}$	Process noise matrix	How much the motion model is distrusted
$\mathbf{R}_{\text{meas}}$	Measurement noise matrix	How much the sensor is distrusted
$\mathbf{K}_{\text{gain}}$	Kalman gain matrix	How much to trust the sensor versus the model
$\mathbf{z}$	Measurement vector	The raw position reading from the sensor
$\mathbf{y}_{\text{res}}$	Residual vector	The difference between measurement and prediction
$\hat{\mathbf{s}}$	Predicted state	State estimate before incorporating the new measurement
$\mathbf{P}_{\text{cov}}^{-}$	Predicted covariance	Uncertainty estimate before incorporating the new measurement
$\top$	Transpose	Flipping rows and columns of a matrix
-1	Matrix inverse	The matrix equivalent of dividing

What is the transpose $\top$?

Transposing a matrix flips it so rows become columns:

$$\mathbf{v} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad \Rightarrow \quad \mathbf{v}^\top = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

State Vector

Everything the filter tracks is packed into one vector called the **state**:

$$\mathbf{s} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix}$$

- (x, y, z) - position of the object in 3D space (meters)
- (v_x, v_y, v_z) - velocity of the object along each axis (meters per second)

The filter estimates this vector at every frame.

Covariance Matrix $\mathbf{P_{cov}}$

The filter does not just track the state - it also tracks **how uncertain it is** about the state. This is stored in $\mathbf{P}_{\text{cov}}$, a 6×6 matrix.

- Diagonal entries = uncertainty about each individual state variable
- Off-diagonal entries = whether errors in one variable are correlated with another (e.g. if the x estimate is wrong, is the v_x estimate also likely to be wrong?)

Large $\mathbf{P_{cov}}$ = uncertain. Small $\mathbf{P_{cov}}$ = confident.

Step 0 - Initialization (First Detection)

On the very first detection, the camera provides a position. The state is initialized as:

$$\mathbf{s}_0 = \begin{bmatrix} x_{\text{measured}} \\ y_{\text{measured}} \\ z_{\text{measured}} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Velocity is set to zero because there is no information about it yet. Initial uncertainty is set to the identity matrix - expressing moderate, uniform uncertainty about all variables:

$$\mathbf{P}_{\text{cov},0} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Loop - Two Steps Per Frame

Step 1 - Predict

Before looking at the camera, the filter asks: **where is the object now, based on where it was last frame?**

Transition Matrix $\mathbf{F_{trans}}$ The constant-velocity motion model states: new position = old position + velocity * time. Written as a matrix multiply operation (at 30 Hz, $\Delta t = 1/30$ seconds):

$$\mathbf{F}_{\text{trans}} = \begin{bmatrix} 1 & 0 & 0 & \frac{1}{30} & 0 & 0 \\ 0 & 1 & 0 & 0 & \frac{1}{30} & 0 \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{30} \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The top-right block (Δt values) propagates velocity into position. The bottom-right identity block keeps velocities unchanged.

Predicted State Multiplying the transition matrix by the previous state gives the predicted state:

$$\hat{\mathbf{s}}_t = \mathbf{F}_{\text{trans}} \cdot \mathbf{s}_{t-1}$$

Expanding the matrix multiply:

$$\hat{\mathbf{s}}_t = \begin{bmatrix} x_{t-1} + v_{x,t-1} \cdot \frac{1}{30} \\ y_{t-1} + v_{y,t-1} \cdot \frac{1}{30} \\ z_{t-1} + v_{z,t-1} \cdot \frac{1}{30} \\ v_{x,t-1} \\ v_{y,t-1} \\ v_{z,t-1} \end{bmatrix}$$

Position advances by velocity * time. Velocity is assumed unchanged.

Process Noise Matrix $\mathbf{Q}_{\text{proc}}$ The motion model is not perfect - the object may accelerate, turn or bounce. $\mathbf{Q}_{\text{proc}}$ represents how much the model's own predictions are distrusted:

$$\mathbf{Q}_{\text{proc}} = \begin{bmatrix} 0.0001 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.0001 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.0001 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.01 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.01 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.01 \end{bmatrix}$$

Small values on position rows (0.0001) - position changes predictably via the model. Larger values on velocity rows (0.01) - velocity can change unexpectedly between frames.

Predicted Covariance Because time has passed and the model is imperfect, uncertainty grows. The predicted covariance is computed as:

$$\mathbf{P}^{-}_{\text{cov},t} = \mathbf{F}_{\text{trans}} \cdot \mathbf{P}_{\text{cov},t-1} \cdot \mathbf{F}^{\top}_{\text{trans}} + \mathbf{Q}_{\text{proc}}$$

Step by step:

1. $\mathbf{F}_{\text{trans}} \cdot \mathbf{P}_{\text{cov},t-1}$ - propagates the previous uncertainty forward through the motion model. Result: 6×6.
2. $(\mathbf{F}_{\text{trans}} \cdot \mathbf{P}_{\text{cov},t-1}) \cdot \mathbf{F}^{\top}_{\text{trans}}$ - symmetrizes the result so $\mathbf{P}_{\text{cov}}$ remains a valid covariance matrix. Result: 6×6.
3. $+\mathbf{Q}_{\text{proc}}$ - adds process noise, growing uncertainty to reflect model imperfection. Result: 6×6.

The $^{-}$ superscript means "before incorporating the camera measurement this frame."

Step 2 - Update

The camera provides a new measurement. It is used to correct the prediction.

Measurement Vector z The camera returns a 3D position only:

$$\mathbf{z}_t = \begin{bmatrix} x_{\text{measured}} \\ y_{\text{measured}} \\ z_{\text{measured}} \end{bmatrix}$$

Observation Matrix $\mathbf{H_{obs}}$ The camera measures position only, not velocity. $\mathbf{H}_{\text{obs}}$ extracts the position rows from the full state vector:

$$\mathbf{H}_{\text{obs}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Multiplying $\mathbf{H}_{\text{obs}}$ by the full state vector picks out only the position entries:

$$\mathbf{H}_{\text{obs}} \cdot \hat{\mathbf{s}}_t = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \hat{x}_t \\ \hat{y}_t \\ \hat{z}_t \\ \hat{v}_{x,t} \\ \hat{v}_{y,t} \\ \hat{v}_{z,t} \end{bmatrix} = \begin{bmatrix} \hat{x}_t \\ \hat{y}_t \\ \hat{z}_t \end{bmatrix}$$

This is the predicted position expressed in the same space as the measurement.

Residual $\mathbf{y_{res}}$ - How Wrong Was the Prediction? The residual is the difference between what the sensor measured and what the model predicted:

$$\mathbf{y}_{\text{res},t} = \mathbf{z}_t - \mathbf{H}_{\text{obs}} \cdot \hat{\mathbf{s}}_t = \begin{bmatrix} x_{\text{measured}} - \hat{x}_t \\ y_{\text{measured}} - \hat{y}_t \\ z_{\text{measured}} - \hat{z}_t \end{bmatrix}$$

If the prediction was perfect, $\mathbf{y}_{\text{res},t} = \mathbf{0}$.

Measurement Noise Matrix $\mathbf{R_{meas}}$ The sensor also has noise. $\mathbf{R}_{\text{meas}}$ encodes how much the camera readings are distrusted:

$$\mathbf{R}_{\text{meas}} = \begin{bmatrix} 0.001 & 0 & 0 \\ 0 & 0.001 & 0 \\ 0 & 0 & 0.001 \end{bmatrix}$$

The value 0.001 m^2 corresponds to roughly 3.2 cm standard deviation per axis. This is the baseline default - an arbitrary starting value. The actual ZED camera accuracy from the static experiment is 0.8-1.1 cm standard deviation per axis, which gives calibrated values of approximately 0.000064-0.000121 m^2. Off-diagonal zeros mean measurement errors in x, y and z are assumed independent.

Kalman Gain $\mathbf{K_{gain}}$ - Step by Step The Kalman gain $\mathbf{K}_{\text{gain}}$ decides how much weight to give the sensor versus the model when correcting the state. The full formula is:

$$\mathbf{K}_{\text{gain}} = \mathbf{P}^{-}_{\text{cov},t} \cdot \mathbf{H}^{\top}_{\text{obs}} \cdot \left(\mathbf{H}_{\text{obs}} \cdot \mathbf{P}^{-}_{\text{cov},t} \cdot \mathbf{H}^{\top}_{\text{obs}} + \mathbf{R}_{\text{meas}} \right)^{-1}$$

This is built in five pieces:

Piece 1: $\mathbf{H_{obs}} \cdot \mathbf{P^-_{cov},_t}$

$\mathbf{H}_{\text{obs}}$ is 3×6, $\mathbf{P}^-_{\text{cov},t}$ is 6×6. Result is 3×6.

This extracts only the rows of $\mathbf{P}^-_{\text{cov},t}$ that correspond to position - i.e. which parts of the uncertainty are visible to the sensor.

$$\mathbf{H}_{\text{obs}} \cdot \mathbf{P}^-_{\text{cov},t} = \text{first 3 rows of } \mathbf{P}^-_{\text{cov},t}$$

Piece 2: $\mathbf{H_{obs}} \cdot \mathbf{P^-_{cov},_t} \cdot \mathbf{H^\top_{obs}}$

Take Piece 1 (3×6) and right-multiply by $\mathbf{H}^\top_{\text{obs}}$ (6×3). Result is 3×3.

This also extracts the relevant columns. The result contains only the position-to-position block of the covariance:

$$\mathbf{H}_{\text{obs}} \cdot \mathbf{P}^-_{\text{cov},t} \cdot \mathbf{H}^\top_{\text{obs}} = \begin{bmatrix} P_{xx} & P_{xy} & P_{xz} \\ P_{yx} & P_{yy} & P_{yz} \\ P_{zx} & P_{zy} & P_{zz} \end{bmatrix}$$

This is the prediction uncertainty expressed in the same 3D space as the measurement.

Piece 3: $\mathbf{H_{obs}} \cdot \mathbf{P^-_{cov},_t} \cdot \mathbf{H^\top_{obs}} + \mathbf{R_{meas}}$

Add the sensor noise matrix $\mathbf{R}_{\text{meas}}$ (also 3×3) to Piece 2. Result is 3×3:

$$\mathbf{H}_{\text{obs}} \cdot \mathbf{P}^-_{\text{cov},t} \cdot \mathbf{H}^\top_{\text{obs}} + \mathbf{R}_{\text{meas}} = \begin{bmatrix} P_{xx} + 0.001 & P_{xy} & P_{xz} \\ P_{yx} & P_{yy} + 0.001 & P_{yz} \\ P_{zx} & P_{zy} & P_{zz} + 0.001 \end{bmatrix}$$

This is the **total uncertainty in measurement space** - the combined prediction uncertainty and sensor noise. Call this matrix $\mathbf{S}_{\text{innov}}$ (the innovation covariance).

Piece 4: $\mathbf{S^{-1}_{innov}}$

Invert the matrix from Piece 3.

Intuition: if $\mathbf{S}_{\text{innov}}$ is large (high total uncertainty), then $\mathbf{S}^{-1}_{\text{innov}}$ is small, which will produce a smaller gain - meaning the correction is less aggressive when total uncertainty is high.

Piece 5: $\mathbf{P^-_{cov},_t} \cdot \mathbf{H^\top_{obs}}$

$\mathbf{P}^-_{\text{cov},t}$ is 6×6, $\mathbf{H}^\top_{\text{obs}}$ is 6×3. Result is 6×3.

This extracts the first 3 columns of $\mathbf{P}^-_{\text{cov},t}$ - how each of the 6 state variables (position and velocity) co-varies with the 3 measured position variables. This includes cross-terms: for example, how uncertain v_x is correlated with x:

$$\mathbf{P}^-_{\text{cov},t} \cdot \mathbf{H}^\top_{\text{obs}} = \begin{bmatrix} P_{xx} & P_{xy} & P_{xz} \\ P_{yx} & P_{yy} & P_{yz} \\ P_{zx} & P_{zy} & P_{zz} \\ P_{v_x x} & P_{v_x y} & P_{v_x z} \\ P_{v_y x} & P_{v_y y} & P_{v_y z} \\ P_{v_z x} & P_{v_z y} & P_{v_z z} \end{bmatrix}$$

Final: Multiply Pieces 5 and 4

$$\mathbf{K}_{\text{gain}} = \underbrace{\mathbf{P}^-_{\text{cov},t} \cdot \mathbf{H}^\top_{\text{obs}}}_{6\times3} \cdot \underbrace{\mathbf{S}^{-1}_{\text{innov}}}_{3\times3} = \underbrace{\mathbf{K}_{\text{gain}}}_{6\times3}$$

$\mathbf{K}_{\text{gain}}$ is a 6×3 matrix. It answers: "for each meter of error in the 3D measurement, how much should each of the 6 state variables be corrected?"

Intuition:

$$\mathbf{K}_{\text{gain}} \approx \frac{\text{prediction uncertainty}}{\text{prediction uncertainty} + \text{sensor noise}}$$

- Prediction uncertain + sensor accurate $\Rightarrow$ large $\mathbf{K}_{\text{gain}} \Rightarrow$ trust the sensor, correct strongly
- Prediction confident + sensor noisy $\Rightarrow$ small $\mathbf{K}_{\text{gain}} \Rightarrow$ trust the model, correct weakly

---

State Update The corrected state is obtained by adding the gain-weighted residual to the predicted state:

$$\mathbf{s}_t = \hat{\mathbf{s}}_t + \mathbf{K}_{\text{gain}} \cdot \mathbf{y}_{\text{res},t}$$

Expanding:

$$\begin{bmatrix} x_t \\ y_t \\ z_t \\ v_{x,t} \\ v_{y,t} \\ v_{z,t} \end{bmatrix} = \begin{bmatrix} \hat{x}_t \\ \hat{y}_t \\ \hat{z}_t \\ \hat{v}_{x,t} \\ \hat{v}_{y,t} \\ \hat{v}_{z,t} \end{bmatrix} + \mathbf{K}_{\text{gain}} \cdot \begin{bmatrix} x_{\text{measured}} - \hat{x}_t \\ y_{\text{measured}} - \hat{y}_t \\ z_{\text{measured}} - \hat{z}_t \end{bmatrix}$$

Note: **velocity also gets updated here**, even though it was never directly measured. This happens because $\mathbf{K}_{\text{gain}}$ has rows for velocity - the filter infers a velocity correction from the position error, using the correlation structure stored in $\mathbf{P}_{\text{cov}}$.

Covariance Update After incorporating the new measurement, uncertainty decreases. The updated covariance is:

$$\mathbf{P}_{\text{cov},t} = (\mathbf{I} - \mathbf{K}_{\text{gain}} \cdot \mathbf{H}_{\text{obs}}) \cdot \mathbf{P}^{-}_{\text{cov},t}$$

Step by step:

1. $\mathbf{K}_{\text{gain}} \cdot \mathbf{H}_{\text{obs}}$: $\mathbf{K}_{\text{gain}}$ is 6×3, $\mathbf{H}_{\text{obs}}$ is 3×6. Result is 6×6.
2. $\mathbf{I} - \mathbf{K}_{\text{gain}} \cdot \mathbf{H}_{\text{obs}}$: $\mathbf{I}$ is the 6×6 identity matrix. The subtraction produces a scaling matrix with values between 0 and 1. Result is 6×6.
3. $(\mathbf{I} - \mathbf{K}_{\text{gain}} \cdot \mathbf{H}_{\text{obs}}) \cdot \mathbf{P}^{-}_{\text{cov},t}$: Multiplying shrinks $\mathbf{P}^{-}_{\text{cov},t}$ - uncertainty decreases because new information was received. Result is 6×6.

Large $\mathbf{K}_{\text{gain}}$ (trusting sensor more) produces a larger reduction in uncertainty.

Concrete Example - One Frame

The object was at $x = 1.0$ m with $v_x = 0.6$ m/s last frame.

Predict:

$$\hat{x}_t = 1.0 + 0.6 \cdot \frac{1}{30} = 1.02 \text{ m}$$

Camera measures: $x_{\text{meas}} = 1.04$ m

Residual:

$$y_{\text{res}} = 1.04 - 1.02 = 0.02 \text{ m}$$

Assume $K_x = 0.8$ (scalar gain for this axis, trusting sensor more than model):

State update:

$$x_t = \hat{x}_t + K_x \cdot y_{\text{res}} = 1.02 + 0.8 \cdot 0.02 = 1.036 \text{ m}$$

The final estimate (1.036 m) falls between the prediction (1.02 m) and the measurement (1.04 m), but closer to the measurement because the gain was high.

What Happens When the Object is Not Detected?

If the detector misses the object (up to `MAX_MISS = 5` frames), there is no measurement available. The update step is skipped and only the predict step runs - the filter keeps estimating position from velocity alone. This is called **coasting.**

Forward Prediction

After the filter updates, the estimated future position can be computed directly from the current state using constant-velocity extrapolation - no filter involved:

$$\hat{p}_{t+h} = \begin{bmatrix} x + v_x \cdot h \\ y + v_y \cdot h \\ z + v_z \cdot h \end{bmatrix}, \quad h = 0.2 \text{ s}$$

Note: this does not update $\mathbf{P}_{\text{cov}}$, so it does not account for growing uncertainty over the prediction horizon. This is a known limitation.

The Full Loop

$$\boxed{\mathbf{s}_{t-1},\ \mathbf{P}_{\text{cov},t-1}} \xrightarrow{\text{Predict: } \mathbf{F}_{\text{trans}},\ \mathbf{Q}_{\text{proc}}} \boxed{\hat{\mathbf{s}}_t,\ \mathbf{P}^-_{\text{cov},t}} \xrightarrow{\text{Update: } \mathbf{z}_t,\ \mathbf{H}_{\text{obs}},\ \mathbf{R}_{\text{meas}},\ \mathbf{K}_{\text{gain}}} \boxed{\mathbf{s}_t,\ \mathbf{P}_{\text{cov},t}}$$

Repeat every frame.

Summary Table

Step	What Happens	Key Equation
Init	Set state from first detection	$\mathbf{s}_0 = [x, y, z, 0, 0, 0]^\top$
Predict state	Use velocity to estimate next position	$\hat{\mathbf{s}}_t = \mathbf{F}_{\text{trans}} \cdot \mathbf{s}_{t-1}$
Predict covariance	Uncertainty grows due to model imperfection	$\mathbf{P}^-_{\text{cov},t} = \mathbf{F}_{\text{trans}} \mathbf{P}_{\text{cov},t-1} \mathbf{F}^\top_{\text{trans}} + \mathbf{Q}_{\text{proc}}$
Residual	How wrong was the prediction?	$\mathbf{y}_{\text{res},t} = \mathbf{z}_t - \mathbf{H}_{\text{obs}} \hat{\mathbf{s}}_t$
Innovation covariance	Total uncertainty in measurement space	$\mathbf{S}_{\text{innov}} = \mathbf{H}_{\text{obs}} \mathbf{P}^-_{\text{cov},t} \mathbf{H}^\top_{\text{obs}} + \mathbf{R}_{\text{meas}}$
Kalman gain	How much to trust sensor vs model?	$\mathbf{K}_{\text{gain}} = \mathbf{P}^-_{\text{cov},t} \mathbf{H}^\top_{\text{obs}} \mathbf{S}^{-1}_{\text{innov}}$
Update state	Correct prediction using measurement	$\mathbf{s}_t = \hat{\mathbf{s}}_t + \mathbf{K}_{\text{gain}} \mathbf{y}_{\text{res},t}$
Update covariance	Uncertainty shrinks after measurement	$\mathbf{P}_{\text{cov},t} = (\mathbf{I} - \mathbf{K}_{\text{gain}} \mathbf{H}_{\text{obs}}) \mathbf{P}^-_{\text{cov},t}$
Coast	No detection - skip update, predict only	-
Forward predict	Where will object be in 0.2 s?	$\hat{p}_{t+h} = p + v \cdot h$